Introduction to Theoretical Computer Science

CS2490 - Session 14: From Automata to REs (cont'd)

Pierre Cagne

Fall 2026

September 18, 2026

Recall from last time

Make an algorithm that:

Recall from last time

Make an algorithm that:

- takes as input a NFA $\mathcal{A}$, and

Recall from last time

Make an algorithm that:

- takes as input a NFA $\mathcal{A}$, and
- outputs a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.

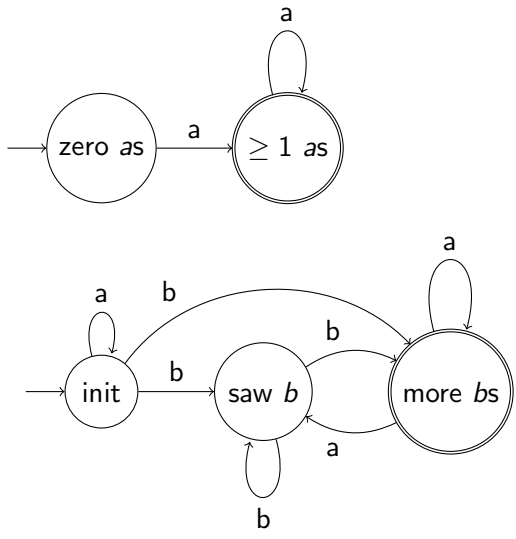
Recall from last time

Make an algorithm that:

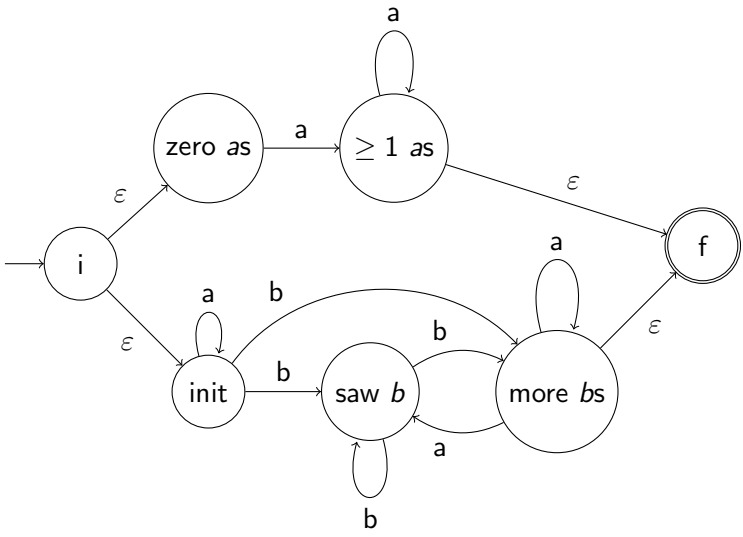
- takes as input a NFA $\mathcal{A}$, and
- outputs a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.

The idea is to gradually remove states from the NFA, while maintaining the same language, until we get to a NFA with only two states and one transition.

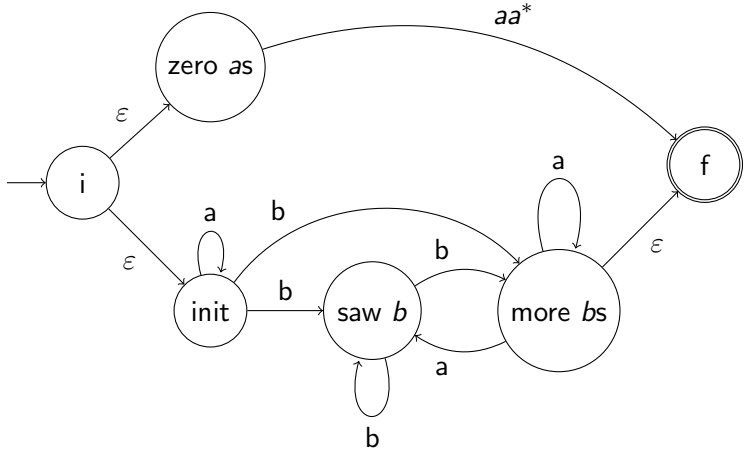
Example from last time



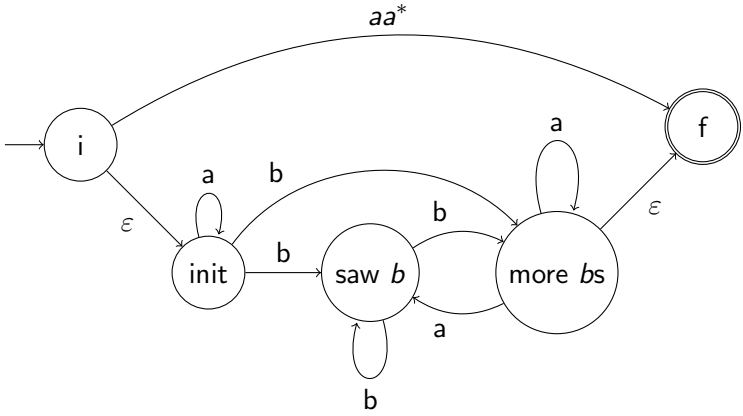
Example from last time



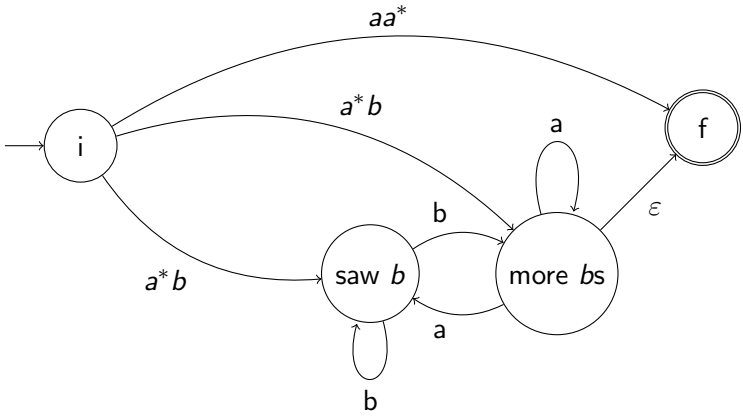
Example from last time



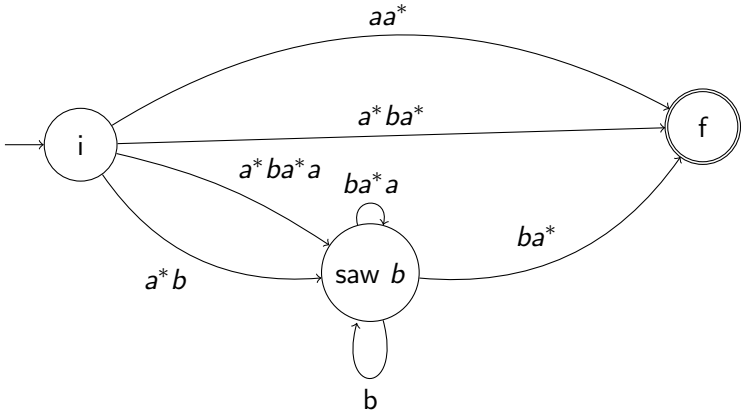
Example from last time



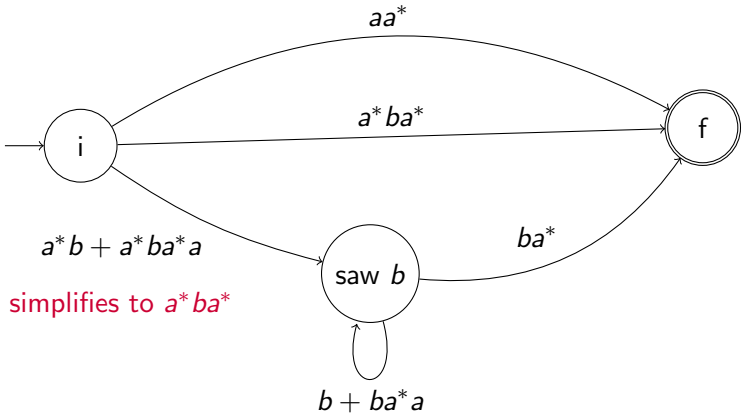
Example from last time



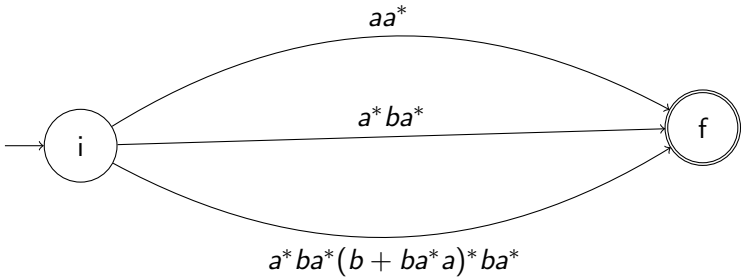
Example from last time



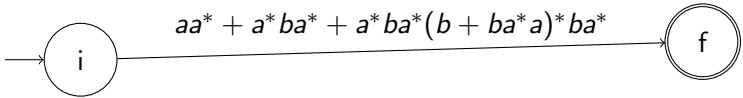
Example from last time



Example from last time



Example from last time



Already noticeable

The RE we get is not necessarily the simplest one.

Already noticeable

The RE we get is not necessarily the simplest one.

The RE we get depends on the order in which we remove states!

Already noticeable

The RE we get is not necessarily the simplest one.

The RE we get depends on the order in which we remove states!

This is of little importance: these REs are usually generated by a program, and are destined to be read by a machine as well.

Algorithm

Let's generalize the steps we performed on the example.

Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

While $Q \neq \{i, f\}$, pick a state $q \in Q \setminus \{i, f\}$ and remove it:

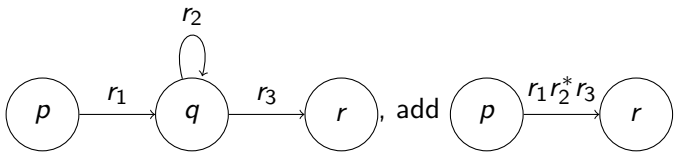
Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

While $Q \neq \{i, f\}$, pick a state $q \in Q \setminus \{i, f\}$ and remove it:

- For each 

.

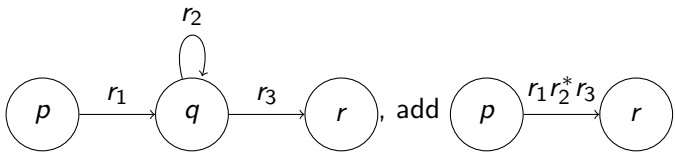
Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

While $Q \neq \{i, f\}$, pick a state $q \in Q \setminus \{i, f\}$ and remove it:

- For each 

- Remove all transitions involving q, then remove q from Q.

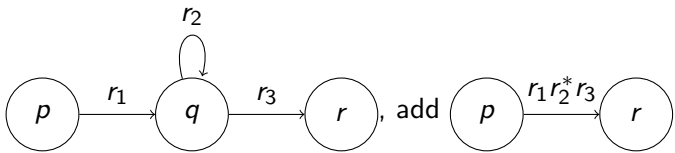
Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

While $Q \neq \{i, f\}$, pick a state $q \in Q \setminus \{i, f\}$ and remove it:

- For each 

.
- Remove all transitions involving q, then remove q from Q.
- Note: r_2 can be ε (if there is no loop on q); p and r can be the same state.

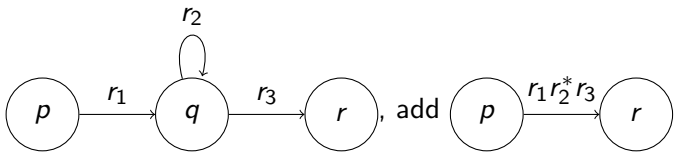
Algorithm

Let's generalize the steps we performed on the example.

Let $\mathcal{A} = (\Sigma, Q, \Delta, I, F)$ be a NFA. We're going to maintain a sequence of generalized NFAs until we get to one with only two states and one transition.

Add a new initial state $i \notin Q$ and a new accepting state $f \notin Q$. Add transitions labeled ε from i to each state in I and from each state in F to f. Set $I = \{i\}$ and $F = \{f\}$.

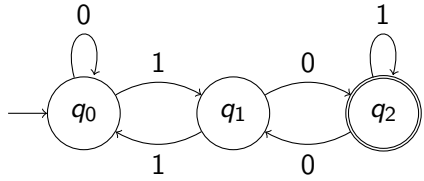
While $Q \neq \{i, f\}$, pick a state $q \in Q \setminus \{i, f\}$ and remove it:

- For each 

.
- Remove all transitions involving q, then remove q from Q.
- Note: r_2 can be ε (if there is no loop on q); p and r can be the same state.
- For any parallel transitions between two states, combine their labels with $+$.

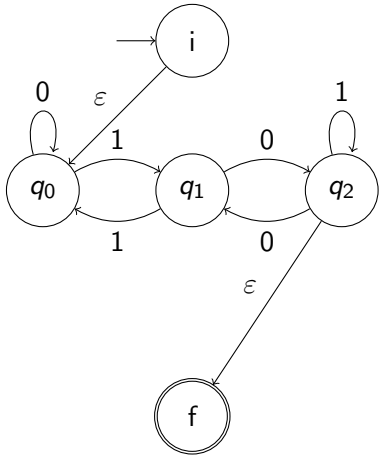
Practice

Find a RE for the language recognized by the NFA below:



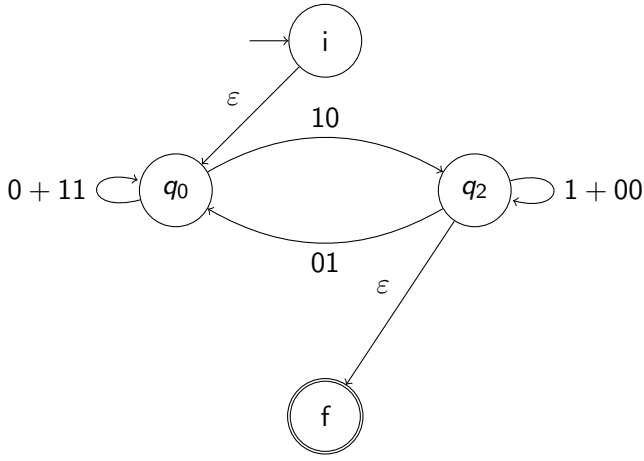
Practice

Find a RE for the language recognized by the NFA below:



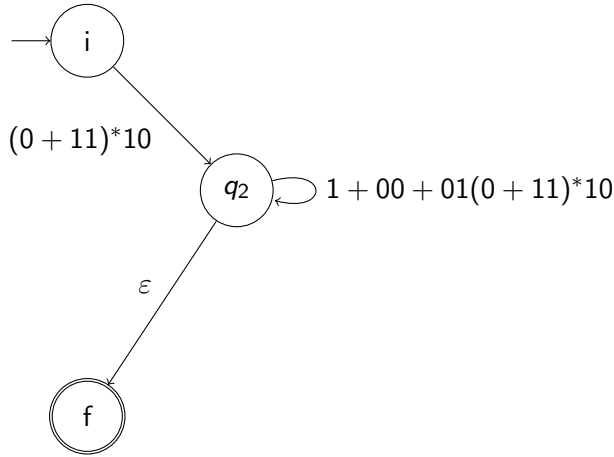
Practice

Find a RE for the language recognized by the NFA below:



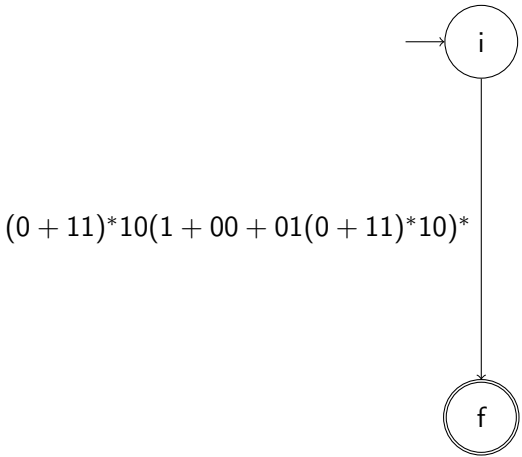
Practice

Find a RE for the language recognized by the NFA below:



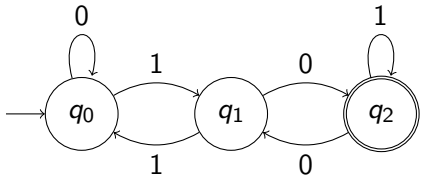
Practice

Find a RE for the language recognized by the NFA below:



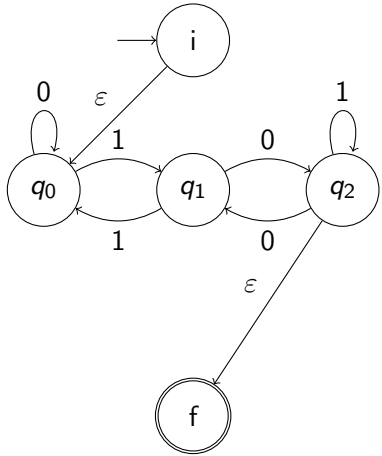
Practice

What if we remove the states in a different order?



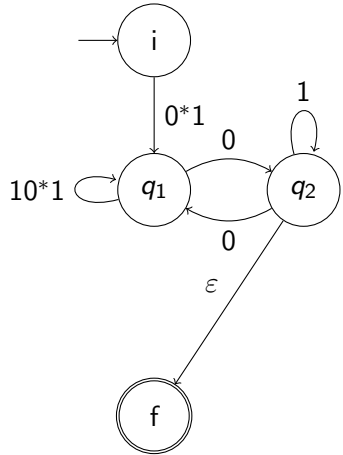
Practice

What if we remove the states in a different order?



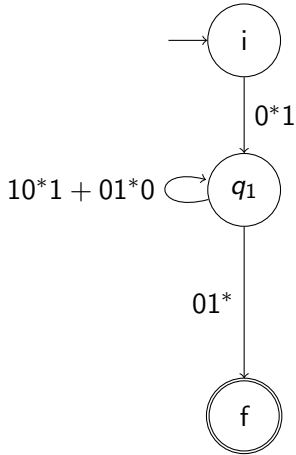
Practice

What if we remove the states in a different order?



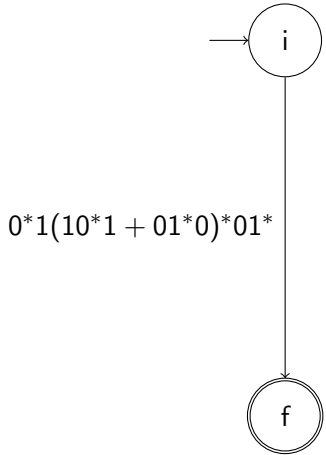
Practice

What if we remove the states in a different order?

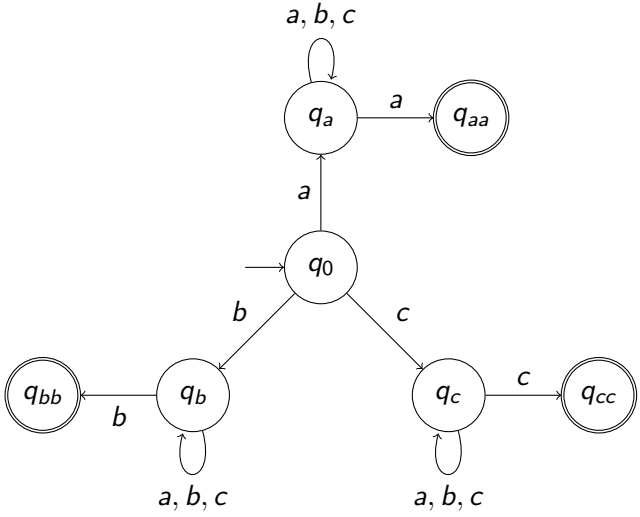


Practice

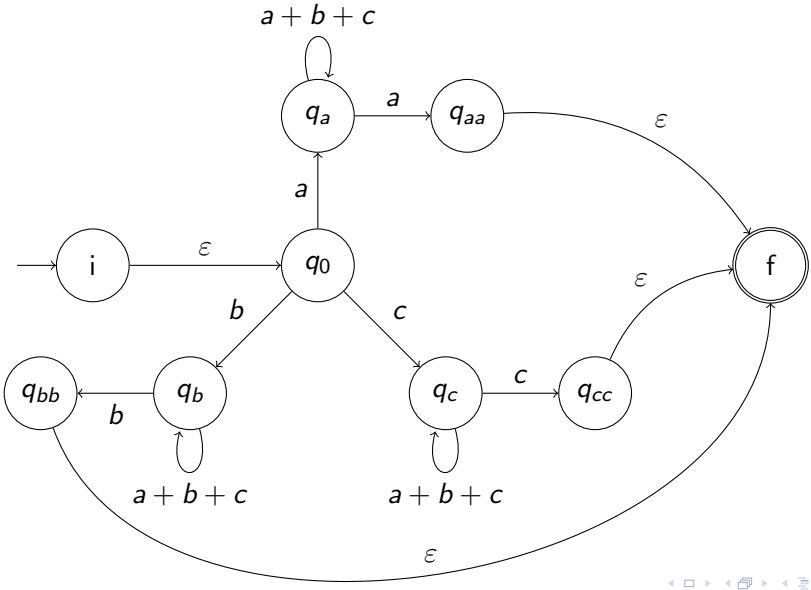
What if we remove the states in a different order?



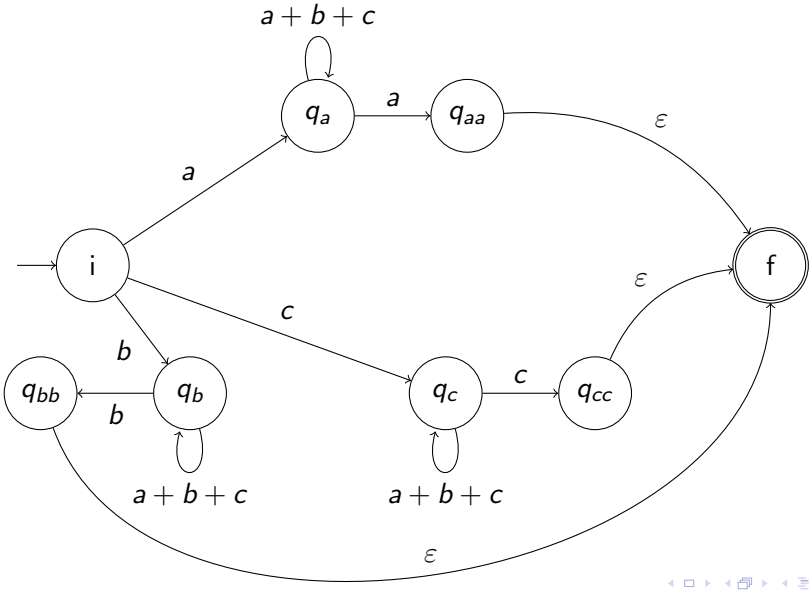
Yet another example



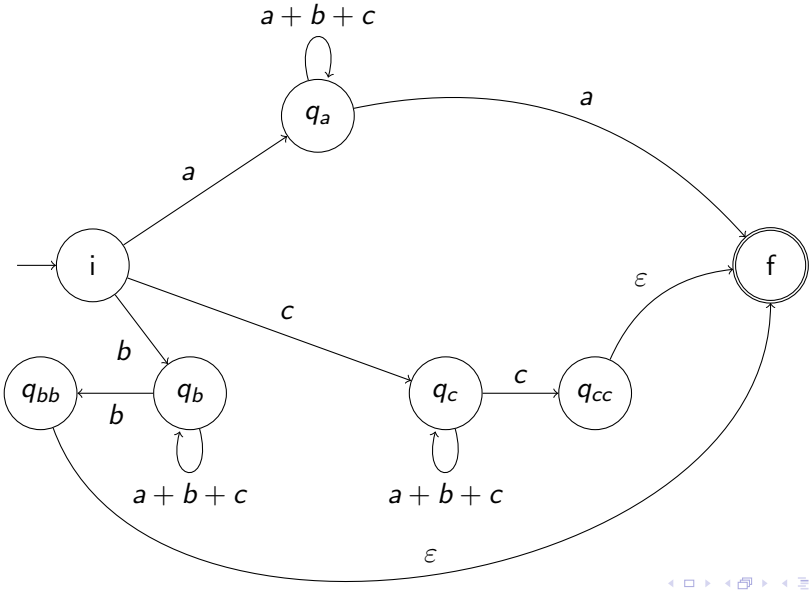
Yet another example



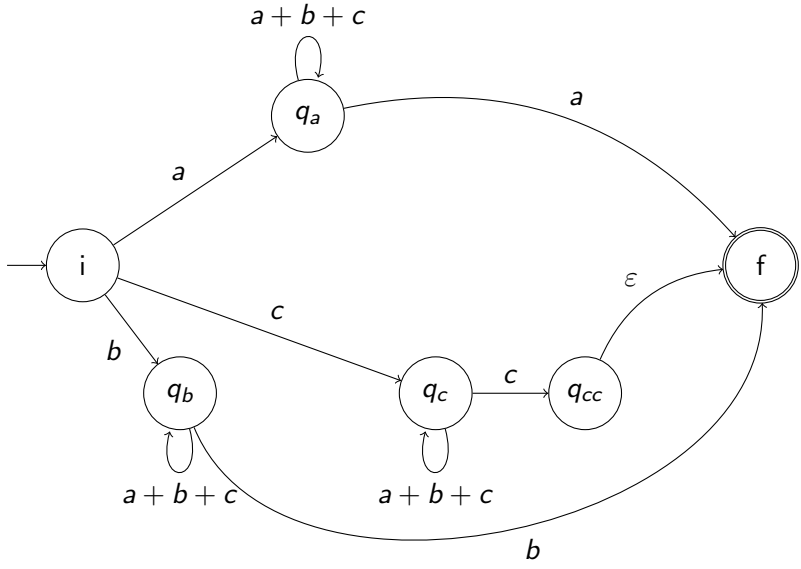
Yet another example



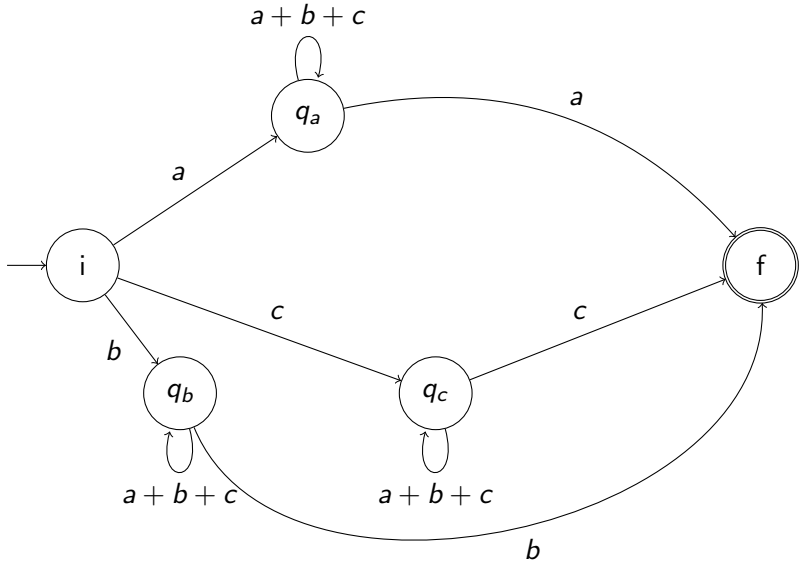
Yet another example



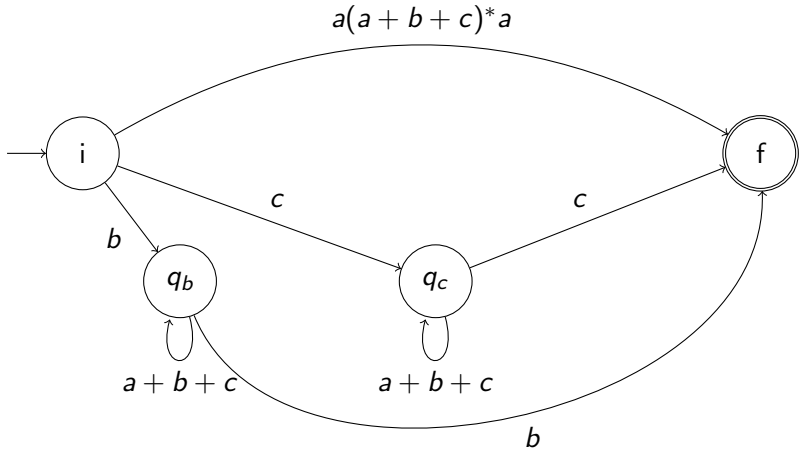
Yet another example



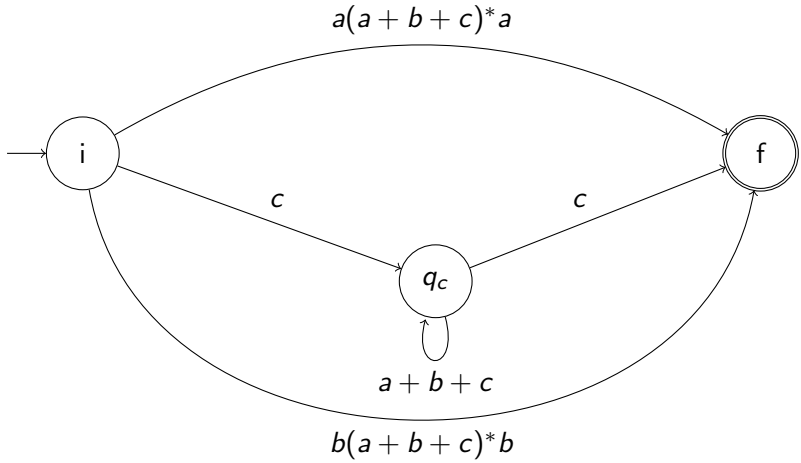
Yet another example



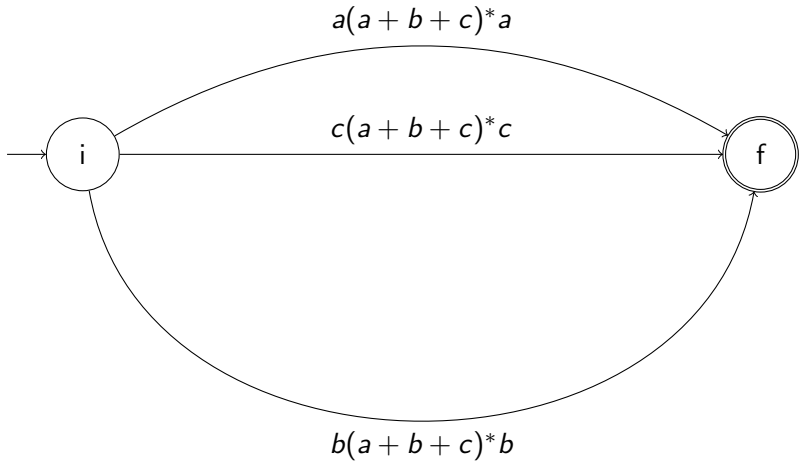
Yet another example



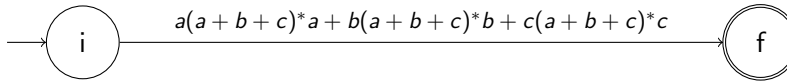
Yet another example



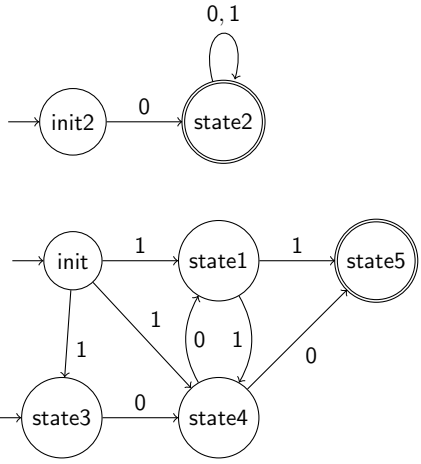
Yet another example



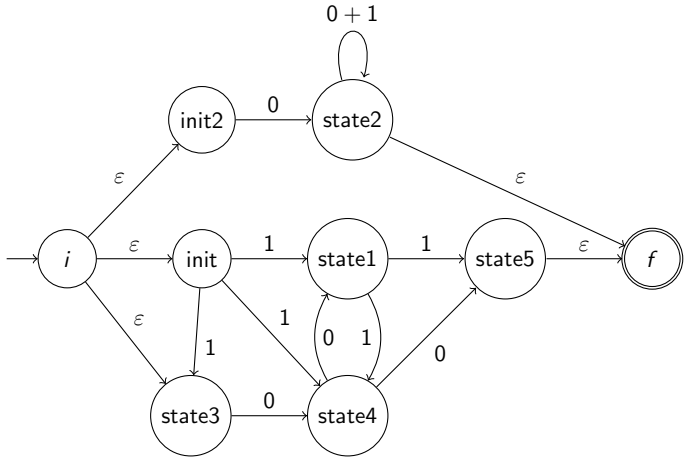
Yet another example



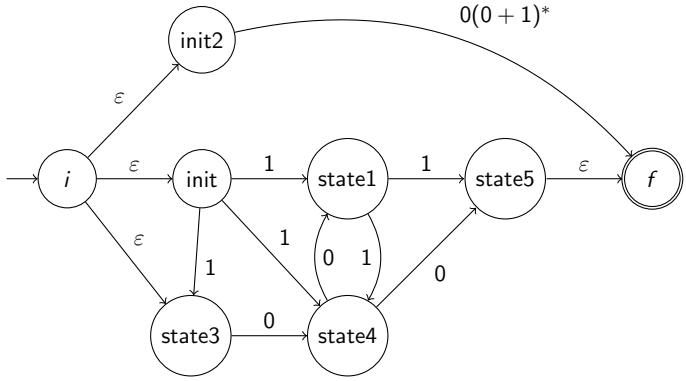
One more example



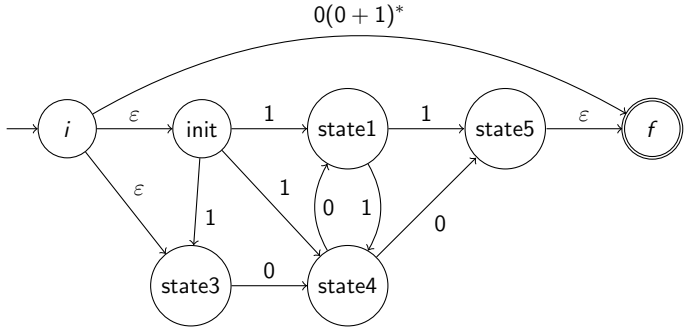
One more example



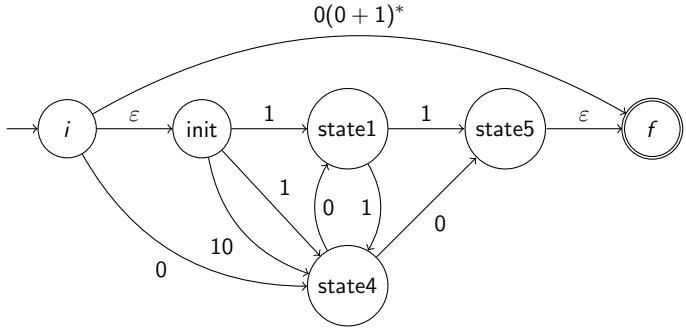
One more example



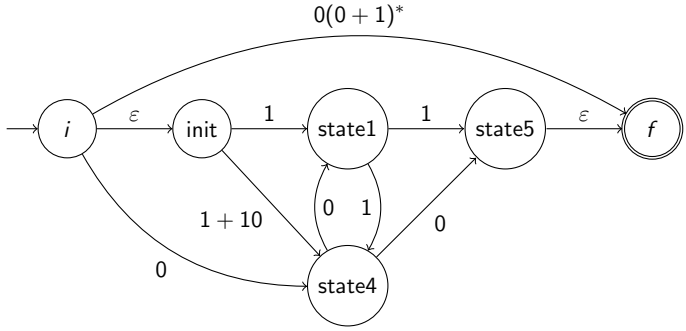
One more example



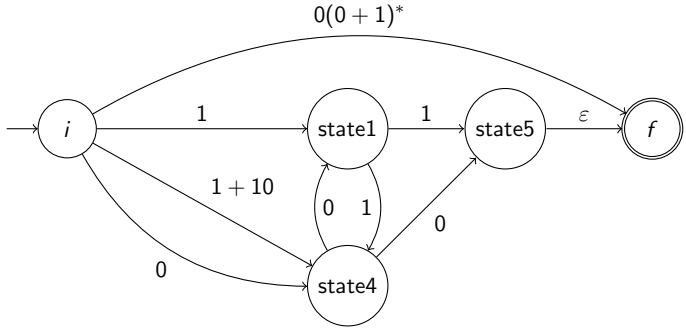
One more example



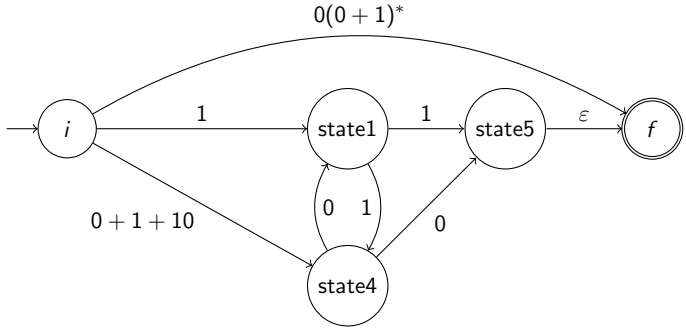
One more example



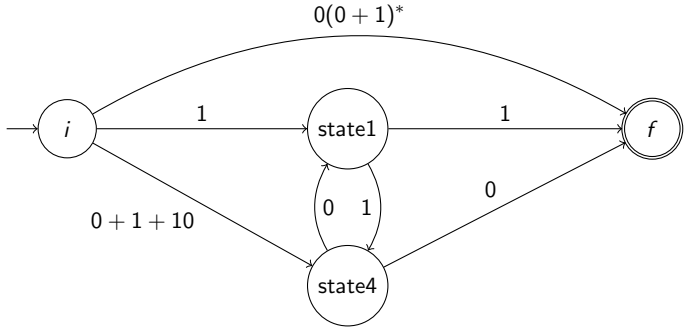
One more example



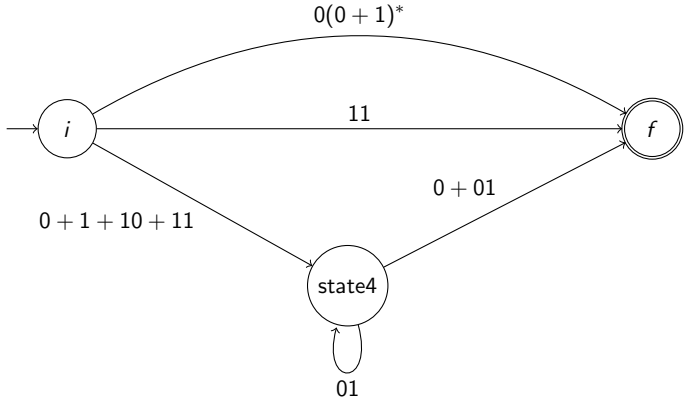
One more example



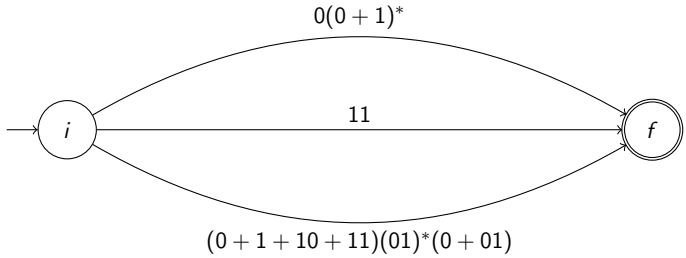
One more example



One more example



One more example



One more example

